Babu Banarasi Das University

SCHOOL OF COMPUTER APPLICATION

BIG DATA FUNDAMENTAL GROUP PROJECT:

Analyzing Cinematic Ratings with Hadoop Ecosystem

Submitted by:

PRACHI PATHAK(1240258316)
PRIYA MAURYA(1240258336)
RAJLAXMI MISHRA(1240258351)
NISTHA GUPTA (1240258301)

Submitted to:

MR. VIKAS (IBM)

***Section*:**

BCADS25

***Date of Submission*:**

25/04/2026

Analyzing Cinematic Ratings with Hadoop Ecosystem

➢ *Architecture: Movie Dataset MapReduce Practical*

This initiative utilizes a conventional Big Data workflow integrated within the Hadoop environment.

Pipeline Overview:

Initial Dataset → HDFS Storage → MapReduce Logic → Structured Output → Hive Querying → Actionable Insights

Component Breakdown:

Data Ingestion: The primary movie CSV is uploaded and held within the HDFS layer.
Processing: A MapReduce framework calculates the mean rating for every unique title.
Storage: Once processed, the aggregated results are saved back into HDFS.
Analytics: Apache Hive is employed to run structured queries on the processed results.
Discovery: Final trends and top-performing films are identified.

Technology Stack:

Storage: Hadoop Distributed File System (HDFS)
Engine: Hadoop Streaming (Python-based MapReduce)
Querying: Hive Data Warehousing
Environment: Linux Command Line
Assistance: Generative AI (ChatGPT)

➢ *Integration of Generative AI*

In this workflow, AI served as a high-level assistant for both technical execution and conceptual clarity.

Primary AI Applications:

1. **Scripting:** Used for drafting the Python-based Mapper and Reducer logic.
2. **Troubleshooting:** Resolved syntax errors and logic bottlenecks during the coding phase.
3. **SQL Optimization:** Assisted in crafting efficient HiveQL scripts for data extraction.
4. **Learning:** Provided deep dives into HDFS nodes and MapReduce shuffle/sort mechanics.

Key Advantages:

- Accelerated development cycles.
- Lowered the frequency of manual syntax errors.
- Enhanced the depth of technical comprehension.

➢ *Dataset Information*

The project utilizes a movie-centric CSV file consisting of audience feedback and film metadata.

Field Definitions:

- `userId`: Unique identifier for the reviewer.
- `movieId`: Catalog ID for the film.
- `title`: The official name of the movie.
- `genre`: The thematic category.
- `rating`: Numerical score (Scale of 1–5).
- `year`: The theatrical release date.
- `watch_time(min)`: Total runtime.
- `location`: Geographic origin of the viewer.
- `review_score`: Detailed quality metric (Scale of 1–10).

	A	B	C	D	E	F	G
1	movieId	title	genres				
2	1	Toy Story	Adventure|Animation|Children|Comedy|Fantasy				
3	2	Jumanji (1	Adventure|Children|Fantasy				
4	3	Grumpier	Comedy|Romance				
5	4	Waiting to	Comedy|Drama|Romance				
6	5	Father of	Comedy				
7	6	Heat (199	Action|Crime|Thriller				
8	7	Sabrina (1	Comedy|Romance				
9	8	Tom and	Adventure|Children				
10	9	Sudden D	Action				
11	10	GoldenEy	Action|Adventure|Thriller				
12	11	American	Comedy|Drama|Romance				
13	12	Dracula: D	Comedy|Horror				
14	13	Balto (199	Adventure|Animation|Children				
15	14	Nixon (19	Drama				
16	15	Cutthroat	Action|Adventure|Romance				
17	16	Casino (19	Crime|Drama				
18	17	Sense and	Drama|Romance				
19	18	Four Roon	Comedy				
20	19	Ace Vent	Comedy				
21	20	Money Tr	Action|Comedy|Crime|Drama|Thriller				
22	21	Get Short	Comedy|Crime|Thriller				
23	22	Copycat (1	Crime|Drama|Horror|Mystery|Thriller				
24	23	Assassins	Action|Crime|Thriller				
25	24	Powder (1	Drama|Sci-Fi				
26	25	Leaving L	Drama|Romance				
27	26	Othello (1	Drama				

Phase 1: Environment Setup

- ***File Creation: Used the cat command to generate the movie.csv locally.***

<u>Step-01</u>: - Create a "CSV" file.

```
[cloudera@quickstart ~]$ cat > movie.csv
userId,movieId,title,genre,rating,year,watch_time(min),device,location,review_score
1,101,KGF,Action,5,2018,150,Mobile,India,9
2,102,RRR,Action,4,2022,180,Laptop,India,8
3,103,Inception,Sci-Fi,5,2010,148,TV,USA,9
4,104,Avengers,Action,4,2012,143,Mobile,UK,8
5,105,Titanic,Romance,5,1997,195,TV,Canada,9
6,101,KGF,Action,3,2018,150,Laptop,India,7
7,102,RRR,Action,5,2022,180,TV,India,9
8,103,Inception,Sci-Fi,4,2010,148,Mobile,USA,8
9,104,Avengers,Action,5,2012,143,Laptop,UK,9
10,105,Titanic,Romance,4,1997,195,TV,Canada,8
11,106,Dangal,Drama,5,2016,161,Mobile,India,9
12,107,3 Idiots,Comedy,5,2009,170,Laptop,India,10
13,108,Interstellar,Sci-Fi,4,2014,169,TV,USA,9
14,109,Bahubali,Action,5,2015,159,Mobile,India,9
15,110,Sholay,Drama,5,1975,204,TV,India,10
16,106,Dangal,Drama,4,2016,161,Laptop,India,8
17,107,3 Idiots,Comedy,5,2009,170,TV,India,10
18,108,Interstellar,Sci-Fi,5,2014,169,Mobile,USA,10
19,109,Bahubali,Action,4,2015,159,Laptop,India,8
20,110,Sholay,Drama,5,1975,204,TV,India,10
21,101,KGF,Action,4,2018,150,Mobile,India,8
22,111,Dhurendhar,Action,5,2025,180,TV,India,9
23,103,Inception,Sci-Fi,5,2010,148,Laptop,USA,9
24,104,Avengers,Action,4,2012,143,Mobile,UK,8
25,105,Titanic,Romance,5,1997,195,TV,Canada,9
26,112,Deewaniyat,Romance,4,2024,160,Mobile,India,8
27,113,OG,Action,5,2024,170,Laptop,India,9
28,114,Saaiyara,Drama,4,2024,150,TV,India,8
29,115,Leo,Action,4,2023,165,Mobile,India,8
30,116,Salaar,Action,5,2023,175,Laptop,India,9
[cloudera@quickstart ~]$
```

- ***HDFS Directory: Created a personal workspace using hdfs dfs -mkdir.***

<u>Step-02:-</u> Create a directory

- hdfs dfs -mkdir /user/cloudera/BD_project
- hdfs dfs -mkdir /user/cloudera/BD_project/movie

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /user/cloudera/BD_projrct
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/BD_project/movie
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/BD_project
Found 2 items
drwxr-xr-x   - cloudera cloudera          0 2026-04-25 03:22 /user/cloudera/BD_project/movie
drwxr-xr-x   - cloudera cloudera          0 2026-04-24 03:56 /user/cloudera/BD_project/spofity
```

Phase 2: Programming the MapReduce Logic

- ***Mapper Development:*** *Created a mapper.py script to parse lines, extract titles, and pair them with ratings.*

```
#!/usr/bin/env python
import sys

for line in sys.stdin:

    if "userId" in line:
        continue

    data = line.strip().split(',')
    movie = data[2]
    rating = float(data[4])
    print("%s\t%s" % (movie, rating))
```


```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    if "userId" in line:
        continue
    data = line.strip().split(',')
    movie = data[2]
    rating = float(data[4])
    print("%s\t%s" % (movie, rating))
```

- ***Reducer Development:*** *Authored reducer.py to aggregate those ratings and compute the final averages per title.*

Step-04:-Createaereducerfileusing“nano”

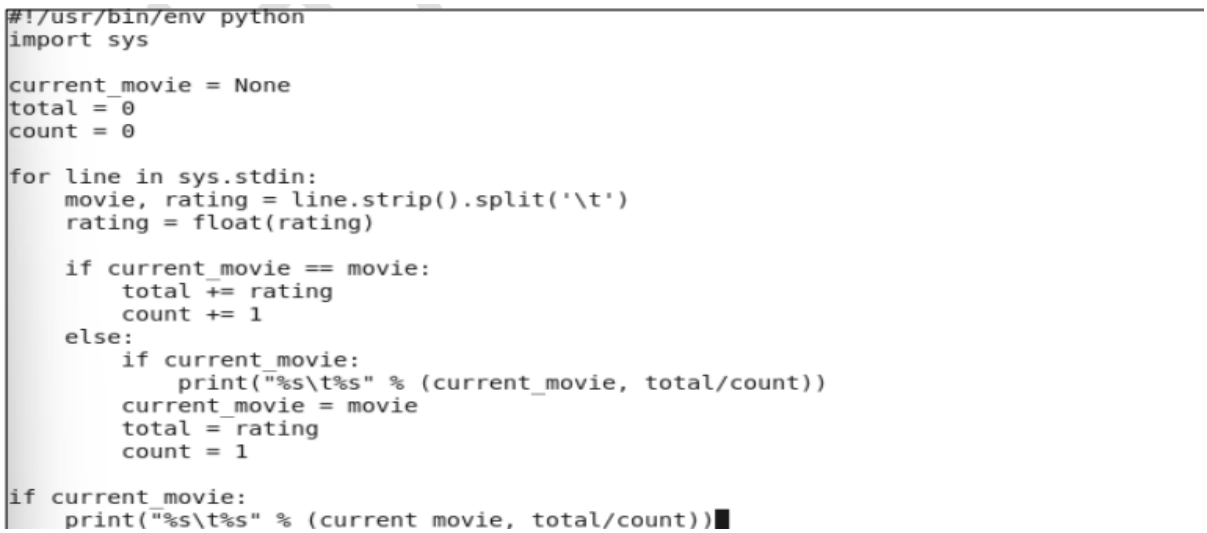
```
#!/usr/bin/env python
import sys

current_movie = None
total = 0
count = 0

for line in sys.stdin:
    movie, rating = line.strip().split('\t')
    rating = float(rating)

    if current_movie == movie:
        total += rating
        count += 1
    else:
        if current_movie:
            print("%s\t%s" % (current_movie, total/count))
        current_movie = movie
        total = rating
        count = 1

if current_movie:
    print("%s\t%s" % (current_movie, total/count))
```



```
#!/usr/bin/env python
import sys

current_movie = None
total = 0
count = 0

for line in sys.stdin:
    movie, rating = line.strip().split('\t')
    rating = float(rating)

    if current_movie == movie:
        total += rating
        count += 1
    else:
        if current_movie:
            print("%s\t%s" % (current_movie, total/count))
        current_movie = movie
        total = rating
        count = 1

if current_movie:
    print("%s\t%s" % (current_movie, total/count))
```

- ***Permission Update: Executed chmod +x to ensure the Python scripts were runnable on the cluster.***

Step-05:- Make executable

- chmod +x mapper.py reducer.py

```
[cloudera@quickstart mapreduce]$ chmod +x mapper.py reducer.py
[cloudera@quickstart mapreduce]$ ls
mapper.py  reducer.py
[cloudera@quickstart mapreduce]$ ls -l
total 8
-rwxrwxr-x 1 cloudera cloudera 220 Apr 20 08:46 mapper.py
-rwxrwxr-x 1 cloudera cloudera 482 Apr 20 08:49 reducer.py
[cloudera@quickstart mapreduce]$
```

Step-06:- Copy “movie.csv” file, from /home/cloudera to /home/cloudera/mapreduce

- cp /home/cloudera/movie.csv /home/cloudera/mapreduce/movie.csv

```
[cloudera@quickstart mapreduce]$ cp /home/cloudera/movie.csv /home/cloudera/mapreduce/movie.csv
[cloudera@quickstart mapreduce]$ ls
mapper.py  movie.csv  reducer.py
[cloudera@quickstart mapreduce]$
```

Phase 3: Execution and Testing

- ***Local Validation: Verified scripts using a standard Linux pipe***

Step-07:-Local test

- cat movie.csv | python mapper.py
- cat movie.csv | python mapper.py | sort | python reducer.py

```
[cloudera@quickstart mapreduce]$ cat movie.csv | python mapper.py
KGF     5.0
RRR     4.0
Inception       5.0
Avengers        4.0
Titanic 5.0
KGF     3.0
RRR     5.0
Inception       4.0
Avengers        5.0
Titanic 4.0
Dangal  5.0
3 Idiots        5.0
Interstellar    4.0
Bahubali        5.0
Sholay  5.0
Dangal  4.0
3 Idiots        5.0
Interstellar    5.0
Bahubali        4.0
Sholay  5.0
KGF     4.0
Dhurendhar      5.0
Inception       5.0
Avengers        4.0
Titanic 5.0
Deewaniyat      4.0
OG      5.0
Saaiyara        4.0
Leo     4.0
Salaar  5.0
[cloudera@quickstart mapreduce]$ cat movie.csv | python mapper.py | sort | python reducer.py
3 Idiots        5.0
Avengers        4.33333333333
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.66666666667
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.66666666667
[cloudera@quickstart mapreduce]$
```

- ***Cluster Execution: Triggered the formal Hadoop job using hadoop-streaming.jar, specifying the input/output paths and the Python scripts.***

Step-08:- Run Hadoop Job

• hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -filesmapper.py,reducer.py -input/user/cloudera/BD_project/movie/movie.csv -output/user/cloudera/BD_project/movie/output -mapper "python mapper.py" -reducer "python reducer.py"

```
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob7469730242140418706.jar tmpDir=null
26/04/20 08:58:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/20 08:58:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/20 08:58:33 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/20 08:58:33 INFO mapreduce.JobSubmitter: number of splits:2
26/04/20 08:58:33 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776698867558_0001
26/04/20 08:58:35 INFO impl.YarnClientImpl: Submitted application application_1776698867558_0001
26/04/20 08:58:35 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_177
6698867558_0001/
26/04/20 08:58:35 INFO mapreduce.Job: Running job: job_1776698867558_0001
26/04/20 08:58:51 INFO mapreduce.Job: Job job_1776698867558_0001 running in uber mode : false
26/04/20 08:58:51 INFO mapreduce.Job:  map 0% reduce 0%
26/04/20 08:59:02 INFO mapreduce.Job:  map 50% reduce 0%
26/04/20 08:59:03 INFO mapreduce.Job:  map 100% reduce 0%
26/04/20 08:59:15 INFO mapreduce.Job:  map 100% reduce 100%
26/04/20 08:59:16 INFO mapreduce.Job: Job job_1776698867558_0001 completed successfully
26/04/20 08:59:16 INFO mapreduce.Job: Counters: 49
```

- ***Result Inspection:*** *Accessed the final output using hdfs dfs -cat on the part-00000 file.*

Step-10:- View Output

▪ hdfs dfs -cat /user/cloudera/ajaychauhan/movie/output/part-00000

```
3 Idiots        5.0
Avengers        4.33333333333
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.66666666667
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.66666666667
[cloudera@quickstart mapreduce]$
```

Phase 4: Hive Analytics

- ***Schema Definition:*** *Formatted a table in Hive (movie_avg) with STRING and FLOAT types to match the MapReduce output.*

Step-1:- Open Hive

- hive

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive

```
CREATE TABLE movie_avg (
   movie STRING,
   avg_rating FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t';
```

```
hive> CREATE TABLE movie_avg (
    >  movie STRING,
    >  avg_rating FLOAT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t';
OK
Time taken: 4.05 seconds
hive> show tables;
OK
count_string
movie_avg
Time taken: 0.382 seconds, Fetched: 2 row(s)
hive>
```

- **Data Migration:** Loaded the HDFS output directory directly into the Hive table.

Step-03:- Load data on the table in movie_avg

LOAD DATA INPATH '/user/cloudera/BD_project/movie/output' INTO TABLE movie_avg;

```
Loading data to table default.movie_avg
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/movie_avg/part-00000': User does no
t belong to hive
Table default.movie_avg stats: [numFiles=1, totalSize=219]
OK
Time taken: 1.471 seconds
hive>
```

Step-04:- View Data

▪ Select * from movie_avg;

```
hive> SELECT * FROM movie_avg;
OK
3 Idiots        5.0
Avengers        4.3333335
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.6666665
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.6666665
Time taken: 0.857 seconds, Fetched: 16 row(s)
hive>
```

- **Insight Retrieval:** Executed `ORDER BY` and `LIMIT` queries to filter the top 5 highest-rated movies based on their mean scores.

Step-05:- Find Top Movies

▪ Select * from movie_avg order by avg_rating desc;

```
hive> SELECT * FROM movie_avg ORDER BY avg_rating DESC;
Query ID = cloudera_20260420090909_04fba5c2-85cf-43ea-aea4-2b7756c7d776
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776698867558_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776698867558
_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776698867558_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-20 09:10:09,363 Stage-1 map = 0%,  reduce = 0%
2026-04-20 09:10:20,037 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 3.05 sec
2026-04-20 09:10:31,713 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 6.39 sec
MapReduce Total cumulative CPU time: 6 seconds 390 msec
Ended Job = job_1776698867558_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 6.39 sec   HDFS Read: 5822 HDFS Write: 207 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 390 msec
OK
OG      5.0
Sholay  5.0
Salaar  5.0
Dhurendhar      5.0
3 Idiots        5.0
Titanic 4.6666665
Inception       4.6666665
RRR     4.5
Interstellar    4.5
Dangal  4.5
Bahubali        4.5
Avengers        4.3333335
KGF     4.0
Saaiyara        4.0
Deewaniyat      4.0
Leo     4.0
Time taken: 40.217 seconds, Fetched: 16 row(s)
hive>
```

Step-06:- Top 5 Movies

- Select * from movie_avg order by avg_rating desc limit 5;

```
hive> SELECT * FROM movie_avg ORDER BY avg_rating DESC LIMIT 5;
Query ID = cloudera_20260420091212_47c4e033-f57b-415a-9f23-cca2fe7d76d5
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776698867558_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776698867558
_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776698867558_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-20 09:13:05,391 Stage-1 map = 0%,  reduce = 0%
2026-04-20 09:13:14,846 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.05 sec
2026-04-20 09:13:26,308 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 5.01 sec
MapReduce Total cumulative CPU time: 5 seconds 10 msec
Ended Job = job_1776698867558_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.01 sec   HDFS Read: 5961 HDFS Write: 57 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 10 msec
OK
OG      5.0
Sholay  5.0
Salaar  5.0
Dhurendhar      5.0
3 Idiots        5.0
Time taken: 35.365 seconds, Fetched: 5 row(s)
hive>
```

➢ Conclusion

This project successfully demonstrated the implementation of a conventional Big Data workflow within the Hadoop environment to analyze cinematic ratings. By building a complete pipeline from initial data ingestion to the extraction of actionable insights, the team effectively processed a dataset consisting of audience feedback and film metadata.

Key Achievements

- **Efficient Data Processing:** A custom Python-based MapReduce framework was successfully deployed via Hadoop Streaming to calculate the mean rating for every unique movie title.
- **Structured Analytics:** Processed data was seamlessly migrated from the Hadoop Distributed File System (HDFS) into Apache Hive. This enabled the execution of structured HiveQL queries to sort and identify the top five highest-rated films.
- **AI-Assisted Development:** The integration of Generative AI significantly accelerated the development cycle. AI served as a high-level assistant for drafting Mapper and Reducer logic, optimizing SQL scripts, and resolving syntax errors during the coding phase.

Final Outcome

The systematic progression through HDFS storage, MapReduce processing, and Hive querying culminated in the successful discovery of final movie trends. The project achieved its objective of transforming raw CSV dataset inputs into organized, queryable intelligence, showcasing practical proficiency in fundamental Big Data technologies.